VITYARTHI PROJECT

RapMap

Desi Hip Hop Song Recommender

Project Report

Course	CSA2001 — Fundamentals in AI and ML
Institute	VIT Bhopal University
Submitted	29 March 2026
Name	Ishan Shrivas
Registration Number	25BAI10966
Branch	Btech. In CSE AI/ML
Session	2025-26
Faculty name	Vaishnavi S.
Github Repo. link	https://github.com/ishan25bai10966/rapmap

1. Project Overview

1.1 What is RapMap?

RapMap is a graph-based song recommendation system built specifically for Desi Hip Hop — a genre that mainstream platforms consistently fail to serve well. The system models songs as nodes in a graph, connects them based on a custom similarity score, and uses three search algorithms — Breadth First Search (BFS), Depth First Search (DFS), and A* — to generate recommendations from any given seed song.

Unlike Spotify or Apple Music, RapMap does not rely on pre-trained Western audio models or aggregated stream data. Instead, it uses a hand-curated dataset of 60 songs from 21 Desi Hip Hop artists, encoding features that actually matter in this genre — language, vibe, BPM range, city scene, and collaboration networks.

1.2 The Problem Being Solved

Mainstream streaming platforms fail at recommending Desi Hip Hop for five distinct reasons:

- Data sparsity — their recommendation models are trained on billions of streams from Western music. Desi Hip Hop has a relatively small global listener base, so the algorithm never sees enough data to understand the genre.
- Mislabeling — artists like Divine, Seedhe Maut, and Emiway are frequently categorized as Bollywood or World Music instead of Hip Hop, causing the algorithm to treat them as unrelated to the genre.
- Language blindness — their Natural Language Processing models are English-trained and cannot parse Hinglish, Haryanvi, or Punjabi slang. Cultural context is lost entirely.
- Cold start problem — new Desi Hip Hop artists with few streams receive almost zero recommendations, creating a cycle where they never grow on the platform.
- Audio feature mismatch — platforms like Spotify use audio features such as tempo and danceability, but their Western-trained models misclassify Indian percussion and melodic patterns.

RapMap addresses this gap by building domain-specific knowledge directly into the graph structure. The similarity function was designed with Desi Hip Hop in mind — city scenes, collab networks, and language nuance are first-class features, not afterthoughts.

1.3 Course Relevance

This project directly applies the following concepts from CSA2001 — Fundamentals in AI and ML:

Course Concept	Application in RapMap
Uninformed Search — BFS	Level-by-level traversal of song graph to find closest recommendations
Uninformed Search — DFS	Deep path exploration to discover niche, unexpected songs
Informed Search — A*	Heuristic-guided search using target vibe to find best match
Problem Solving Agents	The recommender acts as a rational agent with a defined goal state
Knowledge Representation	CSV dataset encodes domain knowledge as structured features
Rational Agents	A* is rational — it always selects the next node with the lowest cost

2. Tools and Technologies

2.1 Programming Language

The entire project is built in Python 3. Python was chosen for its rich ecosystem of data science and graph libraries, its readability, and its suitability for rapid prototyping — all important considerations given the one-week development timeline.

2.2 Libraries

Library	Version	Purpose
pandas	Latest	Reads and cleans the CSV dataset. Converts multi-select fields into lists for comparison.
networkx	Latest	Builds the song graph. Stores nodes and weighted edges. Powers BFS and DFS traversal.
matplotlib	Latest	Visualizes the song similarity graph. Saves output as rapmap_graph.png.
collections.deque	Built-in	Provides an efficient queue for BFS traversal.
heapq	Built-in	Provides a min-heap priority queue for A* search.

2.3 Dataset Tool

The dataset was built collaboratively using Google Sheets, which allowed multiple contributors to fill song data simultaneously using shared dropdown validations. The final sheet was exported as a CSV file (DHH Database.csv) for use in the Python program.

Data validation rules were set up in Google Sheets for the following columns to prevent inconsistencies:

- language — dropdown with multiple selection allowed: Hindi, Urdu, Punjabi, Marathi, English, Hinglish, Haryanvi
- vibe — dropdown with multiple selection allowed: Aggressive, Chill, Conscious, Motivational, Sad
- bpm_range — single selection dropdown: slow, mid, fast
- city — single selection dropdown: Delhi, Mumbai, Pune, Chandigarh, Other
- featured_artists — dropdown matching the artist list to prevent spelling inconsistencies

3. System Requirements

3.1 Software Requirements

Requirement	Specification
Python	Version 3.8 or above
pip	Comes bundled with Python 3.4+
pandas	Install via pip
networkx	Install via pip
matplotlib	Install via pip
Operating System	Windows, macOS, or Linux — any platform supporting Python 3
Terminal	Command Prompt (Windows), Terminal (Mac/Linux)

3.2 Hardware Requirements

RapMap has minimal hardware requirements. Any modern computer capable of running Python 3 is sufficient.

Component	Minimum
RAM	2 GB
Storage	50 MB (for Python, libraries, and dataset)
Processor	Any dual-core processor

3.3 Installation Command

```
pip install pandas networkx matplotlib
```

4. Dataset Design

4.1 Overview

The dataset (DHH Database.csv) contains 60 hand-curated Desi Hip Hop songs from 21 artists. It was built collaboratively using Google Sheets and exported as a CSV file. There was no pre-existing dataset available for this genre with these specific features — the dataset is an original contribution of this project.

4.2 Dataset Schema

Field	Type	Example	Purpose
song_id	Integer	1	Unique identifier for each song
title	Text	Maina	Name of the song
artist	Text	Seedhe Maut	Main artist of the song
language	Multi-select	Hindi, Punjabi	Language(s) used in the song
vibe	Multi-select	Aggressive, Conscious	Mood or feel of the song
bpm_range	Single-select	slow	Tempo category: slow / mid / fast
city	Single-select	Delhi	City the main artist is from
featured_artists	Text	Krsna	Featured artists in the track

4.3 Artists Covered

The dataset covers 21 Desi Hip Hop artists spanning multiple city scenes, languages, and vibes:

City Scene	Artists
Delhi	Seedhe Maut, Krsna, Raftaar, Prabh Deep, Ikka, Encore ABJ, Gravity, Vichaar, Raga, Yashraj
Mumbai	Divine, Emiway, SlowCheeta, Hanumankind, Sambata
Pune	MC Stan
Chandigarh	Sikander Kahlon
Other	Naam Sujal, SOS, Yung Sammy, Siyaahi, Rebel, Karma

4.4 Design Decisions

Several deliberate design decisions were made during dataset construction:

- Special characters were removed from artist names (KR$NA became Krsna) to prevent CSV encoding errors and Python string matching failures.
- Vibe categories were customized for the genre — Conscious and Sad replaced the generic Party and Spiritual labels because they better represent Desi Hip Hop subgenres.
- Haryanvi was added as a language option after discovering that Yung Sammy's songs use it prominently — it is distinct enough from Hindi to affect similarity scoring.
- Multi-selection was enabled for language and vibe because many Desi Hip Hop songs genuinely operate in two languages or two moods simultaneously.
- The year column was removed after analysis — it played no role in the similarity function and would have added noise to the dataset without adding value.

5. System Design and Architecture

5.1 Architecture Overview

RapMap is structured as a four-layer pipeline:

1. Data Layer — DHH Database.csv is loaded and cleaned using pandas.
2. Graph Layer — A NetworkX graph is built where songs are nodes and similarity scores above a threshold become weighted edges.
3. Algorithm Layer — BFS, DFS, and A* traverse the graph from a user-selected seed node.
4. Interface Layer — An interactive terminal menu allows users to select songs, vibes, and algorithms without editing any code.

5.2 Similarity Function

Two songs are connected by an edge if their similarity score is 5 or above. The score is calculated as follows:

Feature Match	Score Added	Reasoning
Same language	+3	Language is a core identity marker in Desi Hip Hop
Same vibe	+3	Mood is the primary driver of listening context
Same BPM range	+2	Similar tempo = similar energy level
Same city	+2	Artists from the same city often share stylistic DNA
Same artist	+4	Songs by the same artist are strongly related
Collaboration	+4	Artists who have worked together are musically connected

The maximum possible score is 20. The minimum threshold for an edge is 5. This threshold was tuned during testing — values below 4 created an overconnected graph (400+ edges), while values above 6 created an underconnected graph (fewer than 50 edges). A threshold of 5 produced approximately 180-200 edges for 60 songs, which is a healthy, well-connected graph.

5.3 Graph Properties

The resulting graph has the following approximate properties:

Property	Value
Total nodes (songs)	60
Total edges (connections)	~180-200
Average connections per song	~6
Most connected song	Highly collaborative songs with shared artist/collab features
Isolated nodes	None — every song connects to at least one other

5.4 Search Algorithms

BFS — Breadth First Search

BFS explores the graph level by level, starting from the seed song. At each level, it visits all neighbours before going deeper. Neighbours are sorted by edge weight in descending order so the strongest connections are explored first. BFS returns songs that are closest in the graph to the seed — these tend to be safe, expected, familiar recommendations.

DFS — Depth First Search

DFS dives deep along one path before backtracking. Unlike BFS, it explores the lowest-weight neighbours first — intentionally going into weaker connections. This causes it to travel far from the seed song, discovering niche and unexpected tracks that BFS would never surface. DFS is the adventurous algorithm — it finds the underground.

A* — A Star Search

A* uses a heuristic function to guide its search toward a target. In RapMap, the target is a user-selected vibe. The heuristic assigns a penalty of 0 to songs that match the target vibe and a penalty of 5 to songs that do not. A* therefore navigates the graph toward songs that match the chosen mood, even if they are not the closest neighbours. It is the most intelligent of the three algorithms — it finds the best match, not just the nearest one.

5.5 Algorithm Comparison

Running all three algorithms on the same seed song produces fundamentally different results. This comparison is the analytical core of the project:

Algorithm	Strategy	Result Type	Best For
BFS	Level-by-level, strongest edges first	Safe, close, expected picks	Users who want familiar recommendations
DFS	Deep path, weakest edges first	Niche, distant, unexpected picks	Users who want to discover underground artists
A*	Heuristic-guided toward target vibe	Mood-matched, intelligent picks	Users who know what feeling they want

6. Application and Features

6.1 Interactive Menu

RapMap runs entirely in the terminal. When launched, it presents a main menu with four options:

5. Get song recommendations — the user picks a seed song, selects a target vibe, and chooses an algorithm (or runs all three for a comparison).
6. Visualize the song graph — displays the full song similarity graph with nodes colored by city scene and the seed song highlighted. Saves the graph as rapmap_graph.png.
7. Show most connected songs — lists the top 5 songs with the most connections in the graph, revealing which songs sit at the center of the Desi Hip Hop ecosystem.
8. Exit — closes the program.

6.2 Song Selection

When requesting recommendations, the user is shown a numbered list of all 60 songs with their titles and artists. They enter the number of the song they want to use as a seed. This means the user never needs to know or edit any song_id — the interface is entirely self-contained and requires no prior knowledge of the dataset.

6.3 Sample Output

```
================================================================
  RAPMAP — Algorithm Comparison
================================================================
  Seed song   : Khatta flow — Seedhe Maut
```

```
  Target vibe : Aggressive
================================================================

  BFS — Closest songs first (safe picks)
  1. Maina — Seedhe Maut                [score: 9]  1 hop(s)
  2. Do guna — Seedhe Maut              [score: 8]  1 hop(s)
  3. Woh raat — Raftaar                 [score: 7]  2 hop(s)
  4. Akatsuki — Seedhe Maut             [score: 7]  2 hop(s)
  5. Nanchaku — Seedhe Maut             [score: 7]  2 hop(s)

  DFS — Deep exploration (niche discoveries)
  1. Blackball — SOS                    [score: 5]  7 hop(s)
  2. New riot — Rebel                   [score: 5]  9 hop(s)
  3. The savarkar rage — Sambata        [score: 5]  11 hop(s)
  4. Naaz — Siyaahi                     [score: 5]  12 hop(s)
  5. Grind — Emiway                     [score: 5]  13 hop(s)

  A* — Smartest match to 'Aggressive' vibe
  1. Woh raat — Raftaar   [score: 7]  why: Aggressive vibe + fast BPM + Delhi
scene
  2. Akatsuki — Seedhe Maut [score: 8] why: Aggressive vibe + fast BPM + Delhi
scene
  3. Jaaluddin — Gravity  [score: 6]  why: Aggressive vibe + fast BPM + Mumbai
scene

  SUMMARY
  BFS → Safe, expected picks close to your seed song
  DFS → Adventurous, niche, unexpected discoveries
  A*  → Smartest match targeting 'Aggressive' vibe
```

6.4 Graph Visualization

The visualization feature generates a spring-layout graph where nodes are positioned based on their connection density. Highly connected songs cluster toward the center. Node colors encode city scene, and edge thickness encodes similarity score — thicker lines indicate stronger connections. The seed song is highlighted with a black outline to make it immediately identifiable.

6.5 Updatable Dataset

Because the entire system reads from DHH Database.csv at runtime, updating the database requires no code changes. Adding new songs is as simple as adding rows to the CSV and re-running the program. The graph rebuilds automatically on each run. This design makes RapMap inherently scalable.

7. Challenges Faced and How They Were Overcome

Challenge 1 — Multi-select fields in CSV

When Google Sheets exports multi-select fields (like "Conscious, Sad") to CSV, they are wrapped in quotes and stored as a single string. Python reads this as one value, not two, meaning the similarity function would fail to match individual vibes.

Solution

A custom parsing function was written to convert these comma-separated strings into Python lists on load. This runs once at startup and converts every multi-select field before the graph is built.

```
def to_list(value):
    if pd.isna(value) or value == '':
        return []
    return [x.strip() for x in str(value).split(',')]
```

Challenge 2 — Special characters in artist names

The artist KR$NA uses a dollar sign in their name. This caused errors in CSV parsing and Python string matching, as the $ character has special meaning in some contexts.

Solution

All artist names with special characters were normalized to plain text equivalents before the dataset was built. KR$NA became Krsna. This was enforced through the Google Sheets dropdown validation, ensuring consistency across the entire dataset. The normalization decision was documented in the dataset design notes.

Challenge 3 — Graph connectivity tuning

The initial similarity threshold of 5 needed to be validated. A threshold that was too low created an overconnected graph where almost every song was connected to every other — making recommendations meaningless. A threshold too high created an underconnected graph where many songs had no edges at all.

Solution

The threshold was tested at values of 4, 5, and 6. A threshold of 5 produced approximately 180-200 edges for 60 songs, with every song having at least one connection. This range was deemed healthy — dense enough for meaningful recommendations but sparse enough that the algorithms had real decisions to make.

Challenge 4 — GUI dependency in matplotlib

The graph visualization used plt.show() which opens a graphical window. This would fail in command-line only evaluation environments that do not support GUI rendering, which would violate the submission requirement for full terminal executability.

Solution

A single line was added at the top of the file to switch matplotlib to a non-interactive backend:

```
import matplotlib
matplotlib.use('Agg')  # saves graph without opening a GUI window
```

This ensures the graph is saved silently as rapmap_graph.png without requiring any graphical display, making the program fully executable in any terminal environment.

Challenge 5 — Dataset consistency across collaborators

With multiple people filling the Google Sheet simultaneously, there was a risk of inconsistent data — different spellings of the same artist, wrong column values, or incorrect featured artist entries. Any inconsistency would break the similarity scoring.

Solution

Data validation dropdowns were configured for every categorical column before sharing the sheet. Featured artists were added to a fixed dropdown to prevent free-text entry. Contributors were briefed with explicit rules — for example, featured artists go only in the featured_artists column, not the artist column. A final review pass was done before exporting to CSV to catch any remaining inconsistencies.

8. Running the Project — Step by Step

8.1 Prerequisites

Ensure the following are installed on your system before proceeding:

- Python 3.8 or above — download from python.org
- pip — comes bundled with Python 3.4 and above

8.2 Step 1 — Clone the Repository

```
git clone https://github.com/ishan25bai10966/rapmap
cd rapmap
```

8.3 Step 2 — Install Dependencies

```
pip install pandas networkx matplotlib
```

This installs all three required libraries. You only need to do this once.

8.4 Step 3 — Verify File Structure

Make sure both files are present in the same folder:

```
rapmap/
├── rapmap.py
├── DHH Database.csv
└── README.md
```

8.5 Step 4 — Update the CSV Path

Open rapmap.py and find line 14. Update the CSV_FILE path to match where you have cloned the project:

```
# Windows example:
CSV_FILE = r'C:\Users\YourName\rapmap\DHH Database.csv'

# Mac / Linux example:
CSV_FILE = 'DHH Database.csv'
```

```
# If your terminal is already inside the rapmap folder:
CSV_FILE = 'DHH Database.csv'
```

8.6 Step 5 — Run the Program

```
python rapmap.py
```

The program runs entirely in the terminal. No GUI setup is required.

8.7 Step 6 — Use the Menu

Once running, you will see the main menu. Enter a number to navigate:

9. Enter 1 to get recommendations. A numbered list of all 60 songs is displayed. Enter the song number, then select a vibe, then select an algorithm. Enter 4 to run all three algorithms and compare their results side by side.
10. Enter 2 to visualize the graph. Select a seed song. The graph opens and is saved as rapmap_graph.png in your project folder.
11. Enter 3 to see the most connected songs in the graph.
12. Enter 4 to exit the program.

9. Results and Analysis

9.1 Graph Analysis

After building the graph from 60 songs, the following observations were made:

- Songs by the same artist cluster tightly together — Seedhe Maut and Krsna songs formed the two densest clusters in the graph.
- Collaboration edges act as bridges between clusters — for example, Raftaar's features on Delhi songs created connections between otherwise distant nodes.
- Songs in Other city had fewer connections due to their geographic diversity, acting as peripheral nodes in the graph.
- Haryanvi songs (Yung Sammy) formed a small isolated cluster due to their unique language combination, which is a realistic reflection of how niche that sub-scene is.

9.2 Algorithm Comparison Results

Running all three algorithms on the same seed song (Khatta flow — Seedhe Maut, target vibe: Aggressive) produced the following contrasting results:

- BFS returned songs that were 1-2 hops away — primarily other Seedhe Maut tracks and Raftaar songs, with high similarity scores (7-9). These are the safe, expected picks.
- DFS traveled 7-13 hops deep, surfacing artists like SOS, Rebel, Sambata, and Siyaahi — none of which would appear in a standard BFS traversal. These are genuinely niche discoveries.
- A* found a different set than BFS despite starting at the same node — it prioritized songs with the Aggressive vibe even when they were not the nearest neighbours, demonstrating the effect of the heuristic.

The fact that all three algorithms return different results from identical input is the central finding of this project — it demonstrates that search strategy is not neutral. The algorithm you choose determines not just what you find, but what kind of listener experience you create.

9.3 Key Insight

The most connected songs in the graph are not necessarily the most popular songs in real life — they are the songs that act as bridges between scenes. A Raftaar track that features both a Delhi and a Mumbai artist, in Hindi, with an aggressive vibe, will connect to more nodes than a solo track with a niche language. This graph centrality analysis reveals something about the genre's internal structure that streaming platform data alone could never show.

10. Limitations

10.1 Dataset Size

60 songs is sufficient to demonstrate the concept but is small by recommendation system standards. A production system would require thousands of songs. The current graph is dense enough to work but not diverse enough to surface truly unexpected recommendations consistently.

10.2 Manual Feature Tagging

All song features — language, vibe, BPM range — were tagged manually by human contributors. This introduces subjectivity. Two people might classify the same song as Chill or Conscious depending on their interpretation. A production system would use audio analysis APIs to extract features objectively.

10.3 No User Feedback Loop

The system does not learn from user behavior. If a user skips a recommendation, that information is lost. A production system would reduce edge weights for skipped songs and increase them for replayed ones, making the graph dynamic over time.

10.4 City Oversimplification

Delhi is overrepresented in the dataset — approximately 35 of 60 songs are from Delhi-based artists. This reflects the real dominance of the Delhi scene in Desi Hip Hop but means that city-based similarity does not add much differentiation in practice.

11. Future Scope

The following enhancements would elevate RapMap from a proof-of-concept to a production-ready system:

- Spotify API Integration — Replace manually tagged BPM with real audio features (tempo, energy, danceability, acousticness) pulled directly from Spotify's API. This would make similarity scoring objective and scalable.
- User Feedback Loop — Track which recommendations users skip or replay. Adjust edge weights dynamically so the graph learns individual preferences over time. Skipped songs lose weight; replayed songs gain weight.
- ML-Based Similarity Scoring — Replace the hand-crafted scoring function with a machine learning model trained on listening history. The model would learn which features actually drive listening behavior rather than relying on manually assigned point values.
- Regional Genre Expansion — Extend the dataset to include Bengali hip hop (Kolkata scene), Tamil street rap, and Bhojpuri rap. Each would require genre-specific vibe categories.
- Web Interface — Build a Flask-based web application that presents the recommendation results and graph visualization in a browser, making the system accessible to non-technical users.
- Community Tagging — Allow fans to contribute vibe and language tags through a crowdsourced interface, improving dataset quality at scale without requiring manual curation.

12. Conclusion

RapMap set out to solve a genuine problem — mainstream streaming platforms consistently fail Desi Hip Hop listeners because their recommendation engines were never designed with this genre in mind. By building a hand-curated dataset, modeling songs as a graph, and implementing three distinct search algorithms, RapMap demonstrates that a domain-specific recommender can surface meaningful connections that general-purpose platforms miss entirely.

The most important finding of this project is not the recommendations themselves — it is the algorithm comparison. BFS, DFS, and A* produce fundamentally different results from identical input. BFS stays safe and close. DFS goes on an adventure. A* homes in on exactly what you asked for. This comparison is a direct, practical demonstration of how search strategy shapes outcome — which is precisely the core insight that CSA2001 aims to convey.

Beyond the academic exercise, RapMap is a system that could genuinely be useful. The dataset is extensible, the graph rebuilds automatically when new songs are added, and the architecture is designed to scale. Every design decision — from the vibe categories to the similarity threshold to the normalization of artist names — was made with the specific cultural context of Desi Hip Hop in mind. That domain knowledge is what makes RapMap different from a generic graph search implementation.

The project was completed within a one-week timeline, covering dataset design and collection, graph construction, algorithm implementation, interactive interface design, visualization, and documentation. The result is a fully executable, well-documented, and genuinely original application of AI search techniques to a real-world problem.

RapMap — Built for CSA2001, VIT Bhopal — 2026